

RISC-V Instruction Set (RV32IM)

<i>instruction</i>	<i>example</i>	<i>meaning</i>
arithmetic		
add	add x5, x6, x7	$x5 = x6 + x7$
subtract	sub x5, x6, x7	$x5 = x6 - x7$
add immediate	addi x5, x6, 2035	$x5 = x6 + \text{sign_extend}(2035)$
set if less than	slt x5, x6, x7	if $(x6 < x7)$, $x5 = 1$ else $x5 = 0$
set if less than immediate	slti x5, x6, 2035	if $(x6 < \text{sign_extend}(2035))$, $x5 = 1$ else $x5 = 0$
set if less than unsigned	sltu x5, x6, x7	if $(x6 < x7)$, $x5 = 1$ else $x5 = 0$
set if < immediate unsigned	sltiu x5, x6, 2035	if $(x6 < 2035)$, $x5 = 1$ else $x5 = 0$
multiply	mul x9, x6, x7	$x9 = (x6 * x7)(31:0)$, lower 32 bits of product
multiply high	mulh x5, x6, x7	$x5 = (x6 * x7)(63:32)$, upper 32 bits of product, both source registers are treated as signed
multiply high signed unsigned	mulhsu x5, x6, x7	$x5 = (x6 * x7)(63:32)$, upper 32 bits of product, x6 is treated as signed, x7 is treated as unsigned
multiply high unsigned	mulhu x5, x6, x7	$x5 = (x6 * x7)(63:32)$, upper 32 bits of product, both source registers are treated as unsigned
divide	div x5, x6, x7	$x5 = x6 / x7$, signed
divide unsigned	divu x5, x6, x7	$x5 = x6 / x7$, unsigned
remainder	rem x9, x6, x7	$x9 = x6 \% x7$, signed
remainder unsigned	remu x9, x6, x7	$x9 = x6 \% x7$, unsigned
upper immediate		
load upper immediate	lui x5, 0x12345	$x5 = 0x12345000$, imm placed in upper 20 bits
add upper immediate to PC	auipc x5, 0x12345	$x5 = PC + 0x12345000$
logic		
and	and x5, x6, x7	$x5 = x6 \& x7$
or	or x5, x6, x7	$x5 = x6 x7$
and immediate	andi x5, x6, 255	$x5 = x6 \& \text{sign_extend}(255)$
or immediate	ori x5, x6, 255	$x5 = x6 \text{sign_extend}(255)$
xor	xor x5, x6, x7	$x5 = x6 \oplus x7$
xor immediate	xori x5, x6, 255	$x5 = x6 \oplus \text{sign_extend}(255)$
shift		
shift left logical immediate	slli x5, x6, 5	$x5 = x6 \ll 5$ (logical)
shift left logical	sll x5, x6, x7	$x5 = x6 \ll x7$ (logical)
shift right logical immediate	srli x5, x6, 5	$x5 = x6 \gg 5$ (logical)
shift right logical	srl x5, x6, x7	$x5 = x6 \gg x7$ (logical)
shift right arithmetic immed	srai x5, x6, 5	$x5 = x6 \gg 5$ (arithmetic), shift in copy of sign bit
shift right arithmetic	sra x5, x6, x7	$x5 = x6 \gg x7$ (arithmetic), shift in copy of sign bit
memory		
load byte	lb x5, 3(x6)	$x5 = \text{memory}[x6+3]$ in least sig. byte, sign extended (sign bit extends 24 bits to left)
load byte unsigned	lbu x5, 3(x6)	$x5 = \text{memory}[x6+3]$ in least sig. byte, zero extended (padded with 24 zeros to the left)
store byte	sb x5, 3(x6)	$\text{memory}[x6+3] = x5$ (byte modified only)
load halfword	lh x5, 2(x6)	$x5 = \text{memory}[x6+2]$ in lower 16 bits, sign extended (sign bit extends 16 bits to left)
load halfword unsigned	lhu x5, 2(x6)	$x5 = \text{memory}[x6+2]$ in lower 16 bits, zero extended (padded with 16 zeros to the left)
store halfword	sh x5, 2(x6)	$\text{memory}[x6+2] = x5$ (only lower 16 bits modified)
load word	lw x5, 8(x6)	$x5 = \text{memory}[x6+8]$
store word	sw x5, 24(x6)	$\text{memory}[x6+24] = x5$
branch		
(immediate branch offset is sign-extended and is in range +/-4KB; may be an instruction label)		
branch if equal	beq x5, x6, 24	if $(x5 == x6)$, $PC += 24$
branch if not equal	bne x5, x6, 24	if $(x5 \neq x6)$, $PC += 24$
branch if less than	blt x5, x6, 24	if $(x5 < x6)$, $PC += 24$; $x5, x6$ treated as signed

branch if greater than or eq	bge x5, x6, 24	if (x5 >= x6), PC += 24; x5, x6 treated as signed
branch if less than unsigned	bltu x5, x6, 24	if (x5 < x6), PC += 24; x5, x6 treated as unsigned
branch if >= unsigned	bgeu x5, x6, 24	if (x5 >= x6), PC += 24; x5, x6 treated unsigned
jump (immediate jump offset is sign-extended; for JAL: range +/-1MB; for JALR: range +/-2KB)		
jump and link	jal x1, 2000	x1 = PC + 4; PC += 2000; if immediate is an instr label, PC = address of labeled instruction
jump and link register	jalr x1, x5, 200	x1 = PC + 4; PC = x5 + 200
environment call (system call)		
environment call	addi a7, x0, 103 ecall	call system routine whose ID is in a7 == 103; registers a0-a7 are used as I/O to the routine

Register	Mnemonic Name	Use
x0	zero	Constant value 0
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5-x7	t0-t2	Temporaries
x8	fp/s0	Frame pointer/saved register
x9	s1	Saved register
x10-x11	a0-a1	Function args/return values
x12-x17	a2-a7	Function arguments
x18-x27	s2-s11	Saved registers
x28-x31	t3-t6	Temporaries